MDPI

*Article*

# Evaluating Parameter-Efficient Fine-Tuning vs. Prompt Engineering for Profile-Driven Recommendations

**Luka Sever [1], Ratomir Karlović [1], Sandi Baressi Šegota [1,*], Vedran Mrzljak [2,*] and Ivan Lorencin [1]**

[1] Faculty of Informatics, Juraj Dobrila University of Pula, Zagrebačka 30, 52100 Pula, Croatia; luka.sever@unipu.hr (L.S.), ratomir.karlovic@unipu.hr (R.K.),sandi.baressi.segota@unipu.hr (S.B.Š.), ivan.lorencin@unipu.hr (I.L.)
[2] Faculty of Engineering Rijeka, University of Rijeka, Vukovarska 58, 51000 Rijeka, Croatia; vedran.mrzljak@riteh.uniri.hr (V.M.)
* Correspondence: sandi.baressi.segota@unipu.hr; vedran.mrzljak@riteh.uniri.hr

**Abstract**

Large language models (LLMs) show significant promise for personalized recommendation systems, yet identifying the most effective adaptation strategy for structured data remains a challenge. This study compares parameter-efficient fine-tuning (PEFT) and in-context learning approaches for personalized point-of-interest recommendation in Pula, Croatia. We utilized explicit user profiles, encompassing specific likes and dislikes, alongside a comprehensive item catalog in JavaScript Object Notation (JSON) format. Two distinct adaptation paradigms were compared on Llama 3.1 8B and Llama 3.2 3B: PEFT, specifically Low-Rank Adaptation (LoRA) and Infused Adapter by Inhibiting and Amplifying Inner Activations ($IA^3$), against prompt engineering strategies optimizing input instructions without altering model weights. Because personalized recommendation is inherently subjective and has no objective ground truth, three human annotators independently judged 1521 recommendations per configuration as agreeing (1), disagreeing ($-1$) or neither (0) with the recommendation, and the consistency of their judgments was quantified using Fleiss' kappa ($\kappa$). Agreement was almost perfect in all six configurations ($\kappa$ between 0.863 and 0.904), indicating a reliable evaluation protocol. Fine-tuned adapters yielded higher agreement than the prompt-engineered baseline for both foundation models, although the differences between configurations were small (at most 0.041), only one of them (LoRA on Llama 3.1 8B) remained statistically significant after correction for multiple comparisons, and no single adapter type was consistently superior across models. These results show that adapter-based fine-tuning produces recommendations whose relevance to explicit user profiles is judged at least as consistently as that of prompt engineering alone.

**Keywords:** large language models; parameter-efficient fine-tuning; prompt engineering; recommender systems; Low-Rank Adaptation (LoRA); structured data processing; in-context learning

Received:
Revised:
Accepted:
Published:

**Copyright:** © 2026 by the authors. Submitted to *Journal Not Specified* for possible open access publication under the terms and conditions of the Creative Commons Attribution (CC BY) license.

## 1. Introduction

The integration of Large Language Models (LLMs) into recommendation systems has fundamentally transformed the field, treating recommendation as a sequence generation task. However, adapting these models to dynamic user preferences presents significant challenges regarding stability and plasticity. To address the continuous evolution of users and items, recent research has explored parameter-efficient fine-tuning (PEFT) methods like Low-Rank Adaptation (LoRA); for instance, the PESO framework maintains a single

evolving adapter anchored by a proximal term to dynamically capture new user behaviors while mitigating the catastrophic forgetting associated with sequential updates [1].

Transitioning LLMs into conversational recommenders introduces additional complexities, particularly concerning semantic misalignment and cumbersome training processes. To streamline this, the PECRS framework reformulates the problem entirely as a natural language processing task, utilizing a frozen pre-trained language model and rich textual metadata to perform both item recommendation and response generation. This unified architecture can be optimized in a single training stage using PEFT techniques, significantly reducing computational overhead [2].

While efficiency is crucial, standard PEFT methods often lack parameter-level personalization because they rely on a static, shared LoRA module for all users. The RecLoRA framework addresses this by introducing a Personalized LoRA module, which dynamically generates unique weights for each user through a soft routing mechanism guided by a conventional recommendation model [3].

Beyond parameter adaptation, prompt engineering and in-context learning offer alternative pathways for efficient LLM integration. Because routing all users through an LLM is computationally prohibitive, recent hybrid task allocation frameworks propose identifying "weak users" with sparse interaction histories and contextualizing their data into natural language instructions. This targeted approach drastically reduces token usage and latency while improving recommendation quality where traditional systems struggle [4].

Finally, applying these adaptations in complex, multi-domain industrial settings requires balancing general user behavior with domain-specific data sparsity. Frameworks such as MLoRA achieve this by combining a shared backbone, which learns generalizable information across domains, with specialized low-rank adaptive modules tailored to capture distinctive, domain-specific features [5].

## 2. Background and Related Work

The integration of Large Language Models (LLMs) into recommender systems has been systematically categorized by Elmoghazy et al., who established a taxonomy based on the LLM's specific role, learning paradigms, and overarching system architecture. Their comparative study emphasizes that while LLMs can act as textual feature encoders or direct recommendation generators, they face persistent challenges such as context window constraints, computational latency, and a disparity between language objectives and recommendation tasks [6]. Further structuring this domain, Bao et al. surveyed the field by dividing architectures into Discriminative and Generative paradigms. Their research details how generative models provide impressive zero-shot capabilities and natural language explainability, despite struggling with context length limits and inherent biases [7]. Additionally, Liu et al. explored language modeling paradigm adaptations, contrasting traditional pre-training with parameter-efficient prompt learning to overcome data sparsity, while noting ongoing issues with factual consistency and user privacy [8].

Expanding on these foundational taxonomies, Nawara and Kashef provided a structured examination of how LLM-powered systems overcome conventional recommendation limitations like data sparsity, cold-start issues, and long-tail challenges. Their survey categorizes recommenders into five distinct paradigms—discriminative, generative, hybrid, graph-enhanced, and multi-modal—while evaluating the transition from traditional metrics to modern generative assessments and "LLM-as-a-Judge" frameworks [9]. Similarly, Shehmir and Kashef conducted a comprehensive systematic literature review of over 150 works to analyze the integration of transformer-based backbones such as Bidirectional Encoder Representations from Transformers (BERT), Generative Pre-trained Transformer

(GPT), and Text-to-Text Transfer Transformer (T5) within recommendation contexts. Their findings map the application of prompt engineering and parameter-efficient fine-tuning (PEFT) across diverse applications while highlighting persistent algorithmic hurdles like high inference latency and societal bias [10].

Shifting the focus specifically toward personalized environments, Chen et al. detailed how the integration of LLMs transforms user experiences from passive filtering into active, conversational, and explainable processes. They established a multifaceted taxonomy where LLMs act as content interpreters, system reasoners, conversational agents, and tool-learning controllers, though they noted significant operational constraints regarding industrial deployment costs and the "phantom" problem of hallucinated content [11]. To frame the technical necessity of tailoring these models, Ling et al. explored the mechanisms of domain specialization required to transform general-purpose LLMs into disruptive tools for highly specific fields. Their systematic framework categorizes adaptation techniques based on model accessibility into black-box external augmentation, grey-box prompt crafting, and white-box model fine-tuning, emphasizing that while fine-tuning offers deep alignment with domain objectives, it introduces risks of catastrophic forgetting and high resource overhead [12].

Transitioning from theoretical surveys to empirical frameworks, Xu et al. proposed LLMRec to systematically evaluate the performance of LLMs functioning directly as recommender systems. Their extensive empirical analysis demonstrated that fine-tuning open-source models via parameter-efficient methods like Low-Rank Adaptation (LoRA) yields superior domain adaptation compared to zero-shot approaches, although targeted reasoning prompts like Chain-of-Thought remain crucial for optimizing prediction accuracy [13].

Focusing specifically on the advancement of parameter-efficient methods, Zhu et al. addressed the inefficiency of static LoRA matrices by introducing RecLoRA. This framework dynamically aggregates multiple independent meta-LoRA weights to create a unique, personalized module for each user, successfully balancing recommendation effectiveness and processing time [3]. Building further on personalized adaptation, Kim et al. identified that standard PEFT foundation models often generate user-blind item embeddings. To resolve this, they proposed PerPEFT, a framework that groups users by similar interests and assigns a distinct, dedicated PEFT module to each demographic. By employing group-specific hard negative sampling, their architecture dynamically captures fine-grained item characteristics, proving highly adaptable to various structures like LoRA and Infused Adapter by Inhibiting and Amplifying Inner Activations ($IA^3$) with minimal parameter overhead [14]. Addressing the dynamic nature of user profiles over time, Zeng et al. tackled the issue of shifting preferences and catastrophic forgetting associated with global fine-tuning. They introduced the Region-Aware Incremental Editing framework, which clusters historical interactions into distinct knowledge regions. When new behaviors emerge, a confidence-aware routing mechanism triggers localized updates to a specialized LoRA module dedicated solely to the affected region, effectively balancing the assimilation of novel interests with the retention of stable historical behaviors [15].

In domain-specific applications, Guo et al. utilized Quantized LoRA within the TIGMI framework to fine-tune an LLM for cultural tourism. This approach reduced computational overhead by 60 percent while successfully integrating Retrieval-Augmented Generation (RAG) for real-time multimodal interactions [16]. Similarly, Choi et al. applied LoRA to the upper layers of a BERT-based model to enhance a RAG framework for autonomous driving, effectively reducing trainable parameters and improving the retrieval of complex legal texts [17]. Furthermore, Gao et al. demonstrated the scalability of fine-tuned LLMs

by developing a multi-dimensional framework to automatically assess the quality and reliability of massive volumes of tourist-generated contents across digital platforms [18].

Conversely, prompt engineering offers an effective alternative that completely avoids altering model weights. Nikolaychuk and Poddubniy applied prompt engineering to the Saiga2 model to automatically extract and classify user sentiments from regional tourism data, successfully achieving high alignment with human expert evaluations [19]. To resolve the complex challenges of intent recognition and factual hallucinations, Huang et al. proposed a multi-layer prompt engineering architecture that decomposes user queries into sequential subtasks. By integrating this step-by-step reasoning strategy with real-time external application programming interfaces (APIs), their system drastically reduced hallucination rates, proving particularly effective for lightweight models in travel planning applications [20].

Based on the cumulative evidence from previous studies, the goal of this work is to examine how adapter-based PEFT compares with prompt engineering strategies for personalized item recommendation. Because personalized recommendation is inherently subjective and lacks an objective ground truth, the study assesses how consistently independent human annotators judge the recommendations produced under each adaptation approach, quantified using Fleiss' kappa. In particular, the study explores how different adaptation approaches, utilizing explicit user profiles and structured JavaScript Object Notation (JSON)-formatted catalogs, influence this agreement. Guided by prior literature, the following hypotheses are considered:

- H1: Annotator judgments of recommendations generated under every adaptation strategy may reach at least substantial agreement ($\kappa > 0.60$), indicating that the annotation protocol is reliable regardless of the adaptation method.
- H2: Recommendations generated by PEFT-adapted models may be judged more consistently (higher $\kappa$) than those generated by the prompt-engineered baseline of the same foundation model, as fine-tuning may align outputs more closely with explicit user preferences.
- H3: LoRA-based adapters may yield higher annotator agreement than other parameter-efficient techniques such as $IA^3$.
- H4: The relative ordering of adaptation strategies may be consistent across foundation models of different size and generation (Llama 3.1 8B and Llama 3.2 3B), indicating that the findings generalize beyond a single architecture.

## 3. Methodology

This section details the experimental framework designed to evaluate and compare the efficacy of parameter-efficient fine-tuning (PEFT) against prompt engineering for personalized point-of-interest (POI) recommendation. We outline the data collection process, the synthesis of user profiles for ground-truth annotation, the retrieval-augmented generation (RAG) pipeline, the specific adaptation strategies applied to the foundation models, and the evaluation protocol based on inter-rater agreement.

### *3.1. System Architecture Overview*

The proposed recommendation system is evaluated through a RAG framework tailored for structured POI data. The architecture deliberately isolates the retrieval mechanism to ensure a controlled, isolated comparison between two distinct Large Language Model (LLM) adaptation paradigms: Prompt Engineering and PEFT. Two foundation models—Llama 3.1 8B and Llama 3.2 3B—are evaluated to assess the scalability, robustness, and architectural generalizability of these adaptation strategies.

### 3.2. Data Collection and Vector Indexing

To simulate a real-world local recommendation engine, a dataset of POIs within the city of Pula, Croatia, was curated via web scraping. The final knowledge base contains 321 POIs of 32 types. Table 1 shows the ten most frequent types. The 126 gastronomy POIs (restaurants, cafes, fast food places and pubs) are the largest group. The remaining POIs include accommodation, cultural heritage sites and recreational areas.

**Table 1.** Number of POIs in the knowledge base by type. The ten most frequent types are listed, and all remaining types are grouped as other.

| Type | POIs | Share of all POIs (%) |
|---|---|---|
| Restaurant | 74 | 23.1 |
| Cafe | 34 | 10.6 |
| Artwork | 24 | 7.5 |
| Fort | 18 | 5.6 |
| Information | 18 | 5.6 |
| Park | 16 | 5.0 |
| Hotel | 16 | 5.0 |
| Fast food | 15 | 4.7 |
| Attraction | 14 | 4.4 |
| Apartment | 14 | 4.4 |
| Other | 78 | 24.3 |
| Total | 321 | 100.0 |

The raw data was formalized into a standardized JSON schema. Each POI is represented as a single JSON object containing a categorical type and a nested properties dictionary. To enrich the semantic density of the dataset and improve personalized retrieval, the following key metadata fields were populated:

- Identification and Contact Fields: The name of the POI for all POIs, and, for gastronomy POIs only, its phone number, opening hours and address (street and house number).
- Taxonomy Tags: An array of descriptors defining the core attributes of the POI, such as cuisine (e.g., mediterranean), dishes (e.g., pizza), dietary options (e.g., vegetarian) and location (e.g., city_center). For some POIs, the tags are stored as a single text string instead of an array.
- Binary and Categorical Constraints: For gastronomy POIs, features such as outdoor_seating, indoor_seating, and price (low, medium or high) that describe the practical constraints of the POI.
- Contextual Narratives: A textual description, a numeric rating and a list of user reviews, to provide the embedding model with nuanced sentiment and atmospheric context.

Not every field is available for every POI. Missing values are stored as null. Tags are available for 295 of the 321 POIs, descriptions for 254, ratings for 229 and user reviews for 222. Ratings are between 2.5 and 5.0. A POI has between one and five reviews, most often three. Among the 126 gastronomy POIs, the price is available for 91, the phone number for 108 and the opening hours for 111.

Each JSON object was treated as an independent document chunk, which gives 321 chunks. To enable dense retrieval, embeddings were generated using the nomic-embed-text:v1.5 model, selected for its strong performance on heterogeneous text structures. The resulting vectors were indexed using Facebook AI Similarity Search (FAISS) to facilitate efficient similarity computations.

### 3.3. Synthetic User Profile Generation and Ground Truth Annotation

Evaluating personalized recommendations necessitates a dataset that accurately models diverse user intents and constraints. We synthetically generated 5000 unique user profiles. Each profile represents a distinct permutation of demographic and behavioral traits, mapped to the JSON schema:

- Demographics: gender, age, and nationality to simulate cultural and generational variance.
- Behavioral Archetypes: travel_style (e.g., backpacker, luxury) and persona (e.g., pet_owner) to establish the user's primary motivation.
- Explicit Preferences: likes and dislikes structured as arrays of venue tags to serve as positive and negative alignment vectors.
- Quantitative Constraints: price_preference and min_rating to bound the acceptable POI thresholds.
- Natural Language Query (input): A conversational prompt synthesized from the profile's constraints (e.g., "Any nice outdoor spots where I can bring my pet?").

To establish a high-quality ground truth for training and evaluation, the Qwen 3.6 35B model was utilized to generate initial recommendation responses for all 5000 queries. Because personalized recommendation is inherently subjective, human annotators were employed to blindly review and validate the relevance of the generated recommendations against the explicit constraints of the user profiles.

Each of the three annotators independently marked every response with 1 (agree), 0 (neither) or $-1$ (disagree). These labels were used to divide the dataset into training, validation and test splits, as described in Section 3.4.

### 3.4. Dataset Composition and Splits

The dataset is synthetic: the 5000 user profiles and their queries were generated and were not collected from real users. The dataset contains 5000 records, one for each synthetic user profile. Each record contains the profile fields (user identifier, name, age, gender, nationality, travel style, persona, likes, dislikes, price preference and minimum rating), the natural language query, the retrieved local information, the initial response generated by Qwen 3.6 35B, and the annotator labels. The retrieved local information is a numbered list of POI records in the JSON format described above, each with its type and its properties. The records were divided into three splits that do not overlap, based on the labels given by the three annotators. Table 2 shows the number of queries and the selection rule for each split.

**Table 2.** Splits of the dataset.

| Split | Queries | Share of dataset (%) | Selection rule | Use |
|---|---|---|---|---|
| Training | 2609 | 52.2 | All three annotators marked 1 | Training of the adapters |
| Validation | 870 | 17.4 | All three annotators marked 1 | Validation of the adapters |
| Test | 1521 | 30.4 | At least one annotator did not mark 1 | Evaluation of the six configurations |

The training and validation splits therefore contain only responses that all three annotators accepted. The test split contains the remaining queries, which are the most challenging and subjective cases in the dataset. All six configurations were evaluated on the test split. The models without adapters did not use the training or validation splits.

### 3.5. Retrieval and Context Injection Pipeline

During inference, a standardized retrieval pipeline is maintained across all evaluated models to isolate the impact of the LLM adaptation strategy. For a given user input, the query is embedded and searched against the FAISS index. The five chunks with the smallest vector distance are retrieved, and these five JSON chunks are injected directly into the LLM prompt.

### 3.6. LLM Adaptation Strategies

The study contrasts a robust Prompt Engineering baseline against two state-of-the-art PEFT methodologies:

- Prompt Engineering (Baseline): The foundation models are augmented with extended, highly structured system instructions. This method relies entirely on in-context learning, leveraging the five retrieved chunks without any underlying weight updates.
- Parameter-Efficient Fine-Tuning (PEFT): The models are fine-tuned on the consensus-driven training set using two distinct adapter mechanisms:
  - Low-Rank Adaptation (LoRA): Freezes the pre-trained model weights and injects trainable rank decomposition matrices into the transformer architecture, allowing the model to learn domain-specific ranking logic.
  - $IA^3$ (Infused Adapter by Inhibiting and Amplifying Inner Activations): A highly efficient alternative that scales inner activations using learned vectors, significantly reducing the number of trainable parameters and computational overhead compared to LoRA.

### 3.7. Adapter Training Configuration

Both adapter types were trained with the same procedure and the same hyperparameters. Each training example was formatted with the chat template of the foundation model and consisted of the system prompt, the user prompt (user profile, user question and retrieved local information) and the target response. The target response was the response generated by Qwen 3.6 35B that all three annotators had marked with 1. The 2609 queries of the training split were used to train the adapters, and the 870 queries of the validation split were used to monitor the training. The adapters were trained with the supervised fine-tuning trainer of the Transformer Reinforcement Learning (TRL) library and the PEFT library. Training used two NVIDIA A100 graphics processing units (GPUs). During training, the foundation models were loaded in bfloat16 precision without quantization. Table 3 lists the settings shared by both adapter types.

**Table 3.** Training settings shared by the LoRA and $IA^3$ adapters.

| Setting | Value |
|---|---|
| Number of epochs | 3 |
| Learning rate | $2 \times 10^{-4}$ |
| Learning rate schedule | Linear decay after warmup |
| Warmup ratio | 0.03 |
| Optimizer | AdamW |
| Weight decay | 0 |
| Gradient clipping (maximum norm) | 1.0 |
| Batch size per GPU | 1 |
| Gradient accumulation steps | 8 |
| Number of GPUs | 2 |
| GPU type | NVIDIA A100 |
| Effective batch size | 16 |
| Maximum sequence length (tokens) | 512 |
| Numerical precision | bfloat16 |
| Gradient checkpointing | Yes |
| Random seed | 42 |

The effective batch size was 16 examples, which is a batch size of 1 on each of the two NVIDIA A100 GPUs multiplied by 8 gradient accumulation steps. The validation loss was calculated every 50 steps, and a checkpoint was saved at the same interval. The checkpoint with the lowest validation loss was used as the final adapter. Table 4 lists the settings that are specific to each adapter type.

**Table 4.** Settings specific to each adapter type.

| Setting | LoRA | $IA^3$ |
|---|---|---|
| Rank | 32 | Not applicable |
| Scaling factor (alpha) | 64 | Not applicable |
| Dropout | 0.05 | Not applicable |
| Bias | None | Not applicable |
| Target modules | q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj | k_proj, v_proj, down_proj |
| Feed-forward modules | Not applicable | down_proj |

LoRA was applied to all attention and feed-forward projection layers of the transformer blocks, with trainable matrices of rank 32. $IA^3$ was applied only to the key, value and down projection layers. It trains one scaling vector for each of these layers, which rescales the outputs of the key and value projections and the input of the down projection, so it has far fewer trainable parameters than LoRA. At inference, the trained adapters were applied to foundation models loaded with 4-bit quantization (Section 3.8).

Table 5 shows the number of trainable parameters, the training time and the final losses for the four adapters. The checkpoint with the lowest validation loss was the last checkpoint for all four adapters.

**Table 5.** Trainable parameters, training time, final training loss and validation loss of the four adapters. The training time is given in minutes.

| Configuration | Trainable parameters | Training time (min) | Training loss | Validation loss |
|---|---|---|---|---|
| Llama 3.1 8B + LoRA | 83,886,080 | 28.5 | 0.088 | 0.102 |
| Llama 3.1 8B + $IA^3$ | 524,288 | 16.2 | 1.385 | 1.421 |
| Llama 3.2 3B + LoRA | 48,627,712 | 24.5 | 0.094 | 0.108 |
| Llama 3.2 3B + $IA^3$ | 286,720 | 12.5 | 1.832 | 1.878 |

Training took between 12.5 and 28.5 minutes. LoRA has about 160 times more trainable parameters than $IA^3$ for Llama 3.1 8B and about 170 times more for Llama 3.2 3B. The loss values of the LoRA adapters (about 0.1) are much lower than those of the $IA^3$ adapters (1.4 and 1.9).

### *3.8. Prompt Format and Inference Setup*

All six configurations received the same system prompt and the same user prompt format. The system prompt instructs the model to act as a tourism assistant that recommends personalized activities based on the interests in the user profile, the current question of the user, and the local information retrieved by semantic search. It also instructs the model to respect strong user preferences and to answer with a short and clear activity recommendation, followed by a brief explanation if needed. The user prompt contains the user profile (name, travel style, persona, likes, dislikes, price preference and minimum rating), the user question, and the retrieved local information as a list of bullet points.

All foundation models were loaded with 4-bit NormalFloat (NF4) quantization and double quantization. For the configurations with adapters, the trained LoRA or $IA^3$ adapter was loaded on top of the quantized foundation model.

### *3.9. Evaluation Protocol and Inter-Rater Agreement*

Six configurations were evaluated: Llama 3.1 8B and Llama 3.2 3B, each under prompt engineering, LoRA and $IA^3$. In the tables and figures, the configurations that use prompt engineering only are named after the foundation model (Llama 3.1 8B and Llama 3.2 3B), and the configurations with adapters carry the adapter name (for example, Llama 3.1 8B + LoRA). Each configuration generated recommendations for the same set of 1521 test queries. The same three annotators rated all recommendations. They were not told which model or adaptation method produced each recommendation. For every recommendation, the three annotators independently assigned one of three labels: 1 (the annotator agrees with the recommendation), $-1$ (the annotator disagrees) or 0 (neither). Since no objective ground truth exists for subjective recommendation quality, the evaluation relies on the consistency of these independent judgments, quantified with Fleiss' kappa [21].

Let $N$ be the number of evaluated recommendations, $n = 3$ the number of annotators, $k = 3$ the number of categories, and $n_{ij}$ the number of annotators who assigned recommendation $i$ to category $j$. The observed agreement for a single recommendation and its mean are

$$P_i = \frac{1}{n(n-1)} \left( \sum_{j=1}^{k} n_{ij}^2 - n \right), \qquad \bar{P} = \frac{1}{N} \sum_{i=1}^{N} P_i. \tag{1}$$

The expected chance agreement is computed from the overall proportion $p_j$ of ratings in each category, and Fleiss' kappa follows as

$$p_j = \frac{1}{Nn}\sum_{i=1}^{N} n_{ij}, \qquad \bar{P}_e = \sum_{j=1}^{k} p_j^2, \qquad \kappa = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}. \tag{2}$$

A value of $\kappa = 1$ indicates perfect agreement, while $\kappa = 0$ indicates agreement no better than chance. Values are interpreted using the scale of Landis and Koch [22]: slight (0.01–0.20), fair (0.21–0.40), moderate (0.41–0.60), substantial (0.61–0.80) and almost perfect (0.81–1.00) agreement. Fleiss' kappa expresses how consistently annotators judge the recommendations, not whether the judgments are correct. Uncertainty was estimated by nonparametric bootstrap, resampling queries with replacement (20000 replicates) and reporting 95% percentile intervals. Nine comparisons were planned: each adapter configuration against the same model without an adapter (four comparisons), LoRA against $IA^3$ for each model (two comparisons), and the same configuration (without adapter, with LoRA and with $IA^3$) on the two foundation models (three comparisons). The differences in kappa were estimated with a paired bootstrap on the same resampled queries. Because nine comparisons were made, the two-sided p-values were adjusted with the Holm method, which reduces the chance of false positive findings when several comparisons are tested together.

## 4. Results and discussion

This section reports the inter-rater agreement for the six configurations on the test set of 1521 queries. Three annotators rated each recommendation, which gives 4563 ratings for each configuration. The results are presented in four parts: the agreement for each configuration, the distribution of the labels, the comparison of the adaptation strategies, and a discussion of the findings.

### *4.1. Agreement for Each Configuration*

Table 6 shows Fleiss' kappa, its 95% confidence interval, the observed agreement $\bar{P}$ and the chance agreement $\bar{P}_e$ for each configuration. Figure 1 shows the kappa values.

**Table 6.** Fleiss' kappa ($\kappa$), 95% bootstrap confidence interval (CI), observed agreement ($\bar{P}$) and chance agreement ($\bar{P}_e$) for the six configurations on the 1521 test queries.

| Configuration | $\kappa$ | 95% CI | $\bar{P}$ | $\bar{P}_e$ |
|---|---|---|---|---|
| Llama 3.1 8B | 0.863 | 0.840 to 0.884 | 0.926 | 0.464 |
| Llama 3.1 8B + LoRA | 0.902 | 0.882 to 0.920 | 0.950 | 0.487 |
| Llama 3.1 8B + $IA^3$ | 0.877 | 0.851 to 0.901 | 0.954 | 0.630 |
| Llama 3.2 3B | 0.875 | 0.854 to 0.895 | 0.930 | 0.439 |
| Llama 3.2 3B + LoRA | 0.888 | 0.869 to 0.906 | 0.937 | 0.435 |
| Llama 3.2 3B + $IA^3$ | 0.904 | 0.882 to 0.924 | 0.961 | 0.597 |

Fleiss' kappa was between 0.863 and 0.904 for all six configurations. All values are in the almost perfect range (0.81 to 1.00) of the scale used in this study. The lower end of the 95% confidence interval was at least 0.84 for every configuration. Hypothesis H1 is therefore supported: the annotators reached at least substantial agreement under every adaptation strategy.

The observed agreement $\bar{P}$ was between 0.926 and 0.961. The chance agreement $\bar{P}_e$ was between 0.435 and 0.630. It is the agreement that would be expected if the annotators gave their labels independently, with the label frequencies of the configuration. Kappa measures how much of the agreement above this level was reached. $\bar{P}_e$ was higher for the

two IA$^3$ configurations (0.630 and 0.597) than for the other four configurations (0.435 to 0.487), because label 1 was more frequent for the IA$^3$ configurations (Section 4.2). For this reason, Llama 3.1 8B + IA$^3$ has the highest observed agreement of the three Llama 3.1 8B configurations (0.954), but a lower kappa (0.877) than Llama 3.1 8B + LoRA (0.902), which has an observed agreement of 0.950.

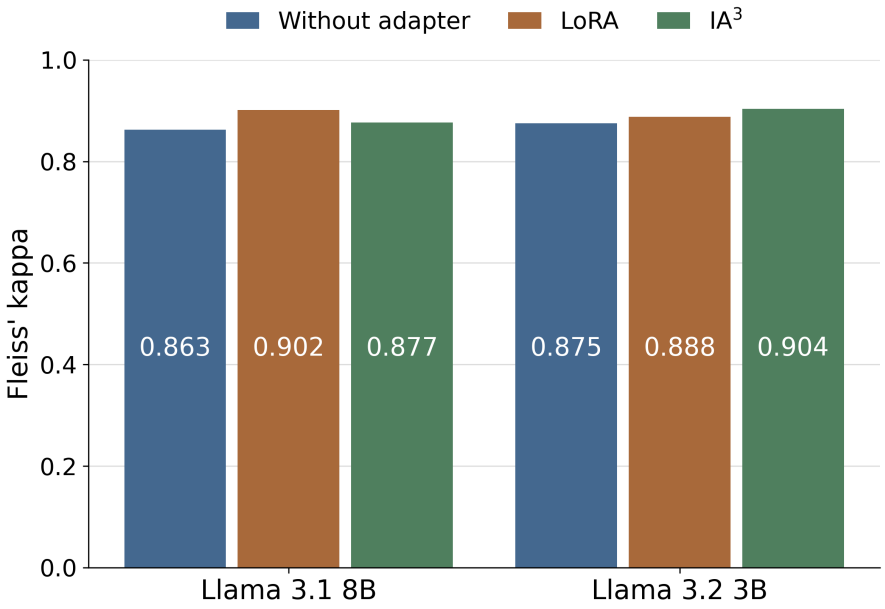



**Figure 1.** Fleiss' kappa for the six configurations.

Figure 1 shows the same kappa values grouped by foundation model. The highest kappa was reached by Llama 3.2 3B + IA$^3$ (0.904), followed closely by Llama 3.1 8B + LoRA (0.902), and the lowest by Llama 3.1 8B without an adapter (0.863). The differences between the configurations are small. The confidence intervals are given in Table 6, and the paired comparisons in Section 4.3 are used to test the differences between configurations, because they take into account that every configuration was rated on the same queries.

### *4.2. Distribution of the Labels*

Table 7 and Figure 2 show how often each label was used. These counts are descriptive. They are reported to show what the annotators decided and to explain the chance agreement in Table 6.

**Table 7.** Number of ratings for each label and, in brackets, their share of the 4563 ratings given to each configuration (three annotators, 1521 queries), in percent.

| Configuration | Label 1 (agree) | Label 0 (neither) | Label −1 (disagree) |
|---|---|---|---|
| Llama 3.1 8B | 2720 (59.6) | 389 (8.5) | 1454 (31.9) |
| Llama 3.1 8B + LoRA | 2862 (62.7) | 346 (7.6) | 1355 (29.7) |
| Llama 3.1 8B + IA$^3$ | 3544 (77.7) | 389 (8.5) | 630 (13.8) |
| Llama 3.2 3B | 2534 (55.5) | 441 (9.7) | 1588 (34.8) |
| Llama 3.2 3B + LoRA | 2449 (53.7) | 417 (9.1) | 1697 (37.2) |
| Llama 3.2 3B + IA$^3$ | 3414 (74.8) | 337 (7.4) | 812 (17.8) |

Label 1 was given to 77.7% of the ratings for Llama 3.1 8B + IA$^3$ and to 74.8% for Llama 3.2 3B + IA$^3$. For the other four configurations, the share of label 1 was between 53.7% and 62.7%. Label −1 showed the opposite pattern: 13.8% and 17.8% for the two

$IA^3$ configurations, and between 29.7% and 37.2% for the other four. Label 0 was used for between 7.4% and 9.7% of the ratings in every configuration. LoRA increased the share of label 1 for Llama 3.1 8B (62.7% compared with 59.6%) but not for Llama 3.2 3B (53.7% compared with 55.5%).

Figure 2 shows the same shares as stacked bars. The share of label 1 (agree) is the longest part of every bar, and the $IA^3$ bars have the smallest part for label −1 (disagree).

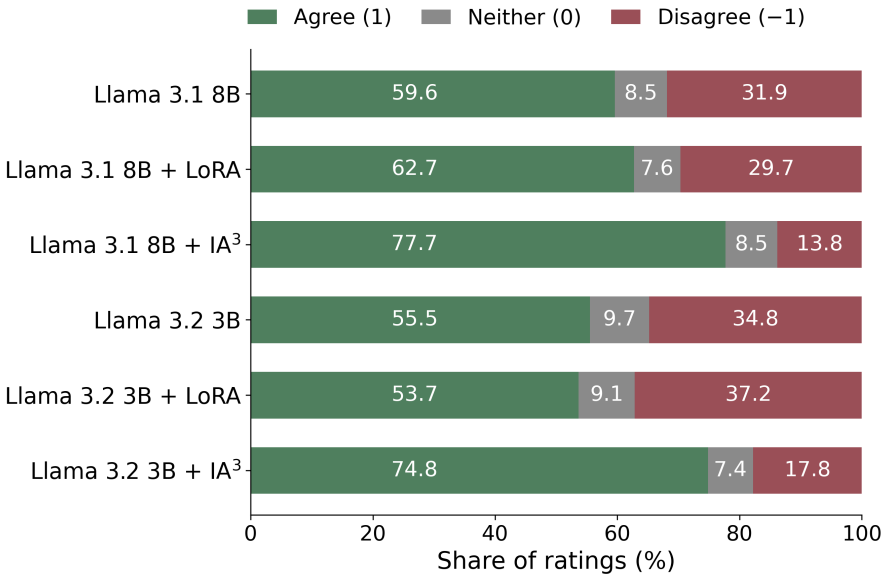



**Figure 2.** Share of the ratings given to each label for the six configurations, in percent.

These shares were not tested statistically, and Fleiss' kappa does not measure them. They show that the annotators agreed with the recommendations of the $IA^3$ configurations more often than with those of the other configurations. This may mean that the $IA^3$ recommendations matched the user profiles better, but this study did not test it.

### *4.3. Comparison of the Adaptation Strategies*

Table 8 shows the nine planned comparisons. Each difference is the kappa of the configuration in the first column minus the kappa of the configuration in the second column.

**Table 8.** Planned comparisons. Difference in Fleiss' kappa between the configuration in the first column and the configuration in the second column, its 95% bootstrap confidence interval (CI) and the p-value adjusted with the Holm method for nine comparisons.

| Configuration | Compared with | Difference | 95% CI | Adjusted p |
|---|---|---|---|---|
| Llama 3.1 8B + LoRA | Llama 3.1 8B | +0.039 | 0.014 to 0.064 | 0.015 |
| Llama 3.1 8B + $IA^3$ | Llama 3.1 8B | +0.014 | −0.016 to 0.044 | 0.878 |
| Llama 3.2 3B + LoRA | Llama 3.2 3B | +0.013 | −0.011 to 0.037 | 0.878 |
| Llama 3.2 3B + $IA^3$ | Llama 3.2 3B | +0.029 | 0.002 to 0.055 | 0.262 |
| Llama 3.1 8B + LoRA | Llama 3.1 8B + $IA^3$ | +0.025 | −0.003 to 0.053 | 0.458 |
| Llama 3.2 3B + LoRA | Llama 3.2 3B + $IA^3$ | −0.016 | −0.034 to 0.003 | 0.458 |
| Llama 3.1 8B | Llama 3.2 3B | −0.012 | −0.037 to 0.012 | 0.878 |
| Llama 3.1 8B + LoRA | Llama 3.2 3B + LoRA | +0.014 | −0.006 to 0.033 | 0.717 |
| Llama 3.1 8B + $IA^3$ | Llama 3.2 3B + $IA^3$ | −0.027 | −0.056 to 0.001 | 0.411 |

Hypothesis H2 expected that the models with adapters would be judged more consistently than the same models without adapters. All four adapter configurations had a higher kappa than the model without an adapter, with differences between +0.013 and +0.039. After the Holm correction, only the difference for LoRA on Llama 3.1 8B was statistically significant (+0.039, 95% CI 0.014 to 0.064, adjusted p = 0.015). The difference for $IA^3$ on Llama 3.2 3B (+0.029) had an unadjusted p-value of 0.033 and an adjusted p-value of 0.262. H2 is therefore supported by the direction of all four differences, but by a statistically significant result for one comparison only.

Hypothesis H3 expected that LoRA would reach a higher agreement than $IA^3$. LoRA had a higher kappa on Llama 3.1 8B (+0.025) and a lower kappa on Llama 3.2 3B (−0.016). Neither difference was statistically significant (adjusted p = 0.458 for both). H3 is not supported.

Hypothesis H4 expected the same order of the strategies for both foundation models. The model without an adapter had the lowest kappa of its three configurations for both Llama 3.1 8B and Llama 3.2 3B. The order of LoRA and $IA^3$ was different for the two models. The same configuration did not differ significantly between the two foundation models: the differences (Llama 3.1 8B minus Llama 3.2 3B) were −0.012 without an adapter, +0.014 with LoRA and −0.027 with $IA^3$, with adjusted p-values between 0.411 and 0.878. The difference between the LoRA and $IA^3$ results was 0.041 larger for Llama 3.1 8B than for Llama 3.2 3B (95% CI 0.008 to 0.074). This comparison was not one of the nine planned comparisons and was not adjusted. H4 is therefore supported for the position of the model without an adapter, but not for the order of the two adapters.

### *4.4. Discussion*

The annotators reached almost perfect agreement for all six configurations. The test set contained only queries for which the annotators had not all marked the initial response with 1 during the ground truth annotation. Even so, the observed agreement for the recommendations of the six configurations was between 0.926 and 0.961. The annotators

did not know which configuration produced each recommendation, so the agreement does not depend on knowledge of the model.

The differences in kappa between configurations were small, at most 0.041. No adapter configuration had a lower kappa than the same model without an adapter. Using LoRA or $IA^3$ therefore did not make the recommendations harder to judge consistently. The only difference that remained statistically significant after correction was LoRA on Llama 3.1 8B. There was no consistent difference between LoRA and $IA^3$, because the better adapter was not the same for the two foundation models. The same configuration also did not differ significantly between Llama 3.1 8B and Llama 3.2 3B.

The results for the $IA^3$ configurations show that kappa should be read together with the label distribution. These configurations received label 1 more often, which raised the chance agreement and lowered kappa relative to the observed agreement. Kappa measures how consistently the annotators judged the recommendations. It does not measure how accurate the recommendations are, so a higher kappa for one configuration does not show that its recommendations are better. The label shares in Table 7 point towards the $IA^3$ configurations as the ones whose recommendations the annotators accepted most often. A dedicated test of the label shares is needed to confirm this.

The label shares also show a result that needs an explanation. The LoRA configurations received label 1 less often than the $IA^3$ configurations, although LoRA trains about 160 to 170 times more parameters. LoRA on Llama 3.2 3B received label 1 less often (53.7%) than the same model without an adapter (55.5%), and label $-1$ more often (37.2% compared with 34.8%). The difference in label 1 is small, 1.8 percentage points, and was not tested statistically, so it may lie within the variation that a single training run can produce. The agreement measured with kappa did not show the same pattern: LoRA had a higher kappa than the model without an adapter for both foundation models. This study did not test the cause of the result, but four explanations follow from its design. They do not exclude each other.

First, the training and test splits come from different parts of the dataset. The training split contains only queries for which all three annotators accepted the response of Qwen 3.6 35B, and the test split contains the queries for which at least one annotator did not accept it. The LoRA adapters reached a training loss of about 0.09 and a validation loss of about 0.1 (Table 5), so they learned to reproduce the responses of Qwen 3.6 35B closely. A model that imitates these responses closely may also repeat the behaviour of Qwen 3.6 35B on the queries for which its responses were not accepted, and these are the queries of the test split. The $IA^3$ adapters reached a much higher loss (1.4 and 1.9), so they did not reproduce the training responses closely. With one scaling vector per layer, they may have learned the general form of an accepted response, such as following the likes and dislikes of the user, without copying the content of individual responses. This could explain why the closer fit of LoRA did not lead to more accepted recommendations on the test split.

Second, the validation split did not protect against this effect. It was selected with the same rule as the training split, so a low validation loss shows that an adapter reproduces accepted responses, but not that it handles the difficult queries of the test split. The checkpoint with the lowest validation loss was the last one for all four adapters, so the checkpoint selection did not stop the training before the closest fit. The training set was also small (2609 queries), and its prompts were built from the same 321 POIs, which makes such a close fit easier to reach.

Third, the share of parameters that LoRA trains is slightly larger for the smaller model. The trainable parameters of the LoRA adapter are about 1.5% of the approximately 3.2 billion parameters of Llama 3.2 3B, compared with about 1.0% of the approximately 8.0 billion parameters of Llama 3.1 8B. The rank, the learning rate and the number of epochs

were the same for both models, so the same adapter setting may have changed the smaller model relatively more. This could be related to the result that LoRA increased the share of label 1 for Llama 3.1 8B (62.7% compared with 59.6%) but not for Llama 3.2 3B (53.7% compared with 55.5%). The difference between 1.5% and 1.0% is small, and the two models differ in other ways as well, so this is the least certain of the four explanations.

Fourth, the adapters were trained on foundation models in bfloat16 precision and applied to the same models with 4-bit quantization (Section 3.8). LoRA adds a learned update to the weights of all seven projection layers, and this update was learned for the weights at full precision. $IA^3$ only rescales the activations of three layers. The difference between the weights used in training and in inference may therefore affect LoRA more than $IA^3$, and the effect of quantization may be larger for the smaller model.

These explanations can be tested in further work (Section 5). The adapters can be evaluated without quantization, the checkpoints can be selected on a validation set that contains difficult queries, LoRA can be trained with a lower rank, a lower learning rate or fewer epochs, each adapter can be trained several times, and the similarity of each configuration's responses to the responses of Qwen 3.6 35B on the test queries can be measured.

Some limits apply to these results. The recommendations are for one city, Pula, and were rated by three annotators. The test set contains only the queries that were difficult during annotation, and the label shares were not tested statistically. These and other limits are discussed in Section 5.

## 5. Limitations and Future Work

This section describes the limitations of the study and proposes directions for future work. The limitations concern the evaluation measure, the data, the models and training, and the settings used for inference. The future work follows from these limitations.

### *5.1. Limitations*

The evaluation in this study is based on Fleiss' kappa, which measures how consistently the three annotators judged the recommendations. A high kappa shows that the annotators agreed with each other. It does not show that a recommendation is correct or that its quality is high. Even when all three annotators mark a response as good, this shows that they agree on it, but it does not measure the quality of the recommendation. Personalized recommendation is subjective, and no objective ground truth exists. The shares of the labels in Table 7 were reported only as descriptive counts and were not tested statistically. For this reason, the study cannot state which configuration gives better recommendations.

Only responses that all three annotators marked with 1 were used for training and validation. This choice made the training data reliable, but it means that the annotation step was not summarized with an agreement statistic. All training responses were also generated by one model, Qwen 3.6 35B, so the adapters learned from the content and style of a single model. The test set of 1521 queries was used to test the configurations after training. It contains the queries for which the annotators did not all mark the initial response with 1, so these queries are more difficult and more subjective than the rest of the dataset. The validation split was selected with the same rule as the training split, so the validation loss used to select the checkpoints shows how well an adapter reproduces accepted responses, but not how well it handles difficult queries. The results therefore show how the configurations behave on difficult cases, and they may be different for a random sample of queries.

The user profiles and queries are synthetic. Real users may write their requests in a different way, with different wording, length and level of detail, so the results may not

transfer directly to real requests. The knowledge base covers one small city, Pula, with 321 POIs, and the system was not tested on a large city with many more POIs. Each POI is stored as one JSON object that contains all of its fields, and this design may not work as well for a much larger number of POIs. The results could be different for larger cities for this reason.

Only two foundation models were used, Llama 3.1 8B and Llama 3.2 3B, and both belong to the same model family. Each adapter was trained once, with one random seed and one set of hyperparameters, which was the same for LoRA and $IA^3$. The settings were not tuned separately for each adapter type, so a different setting could favor one adapter type. The bootstrap confidence intervals describe the variation between test queries. They do not describe the variation between training runs, which can only be measured by training the same adapter several times.

All six configurations were evaluated with 4-bit quantization, while the adapters were trained on models loaded without quantization. Quantization changes the model weights slightly, so an adapter may behave differently on a quantized model than on the model it was trained on. An evaluation without quantization was not done, and the results may be different at full precision. Finally, the study did not measure inference latency or cost. The training time and the number of trainable parameters were reported, but the time and cost needed to generate recommendations were not compared between the configurations.

### *5.2. Future Work*

Several directions follow from these limitations. The first is to test more foundation models, including models from other families and models of different sizes, to check whether the results are specific to the two Llama models. The second is to build knowledge bases for other cities, including large cities with many more POIs, and to include more POI types. This would show whether storing each POI as one JSON object and retrieving five chunks still works at a larger scale.

The third direction is to train each adapter several times with different random seeds, so that the variation between training runs can be measured and reported together with the variation between test queries. The fourth is to evaluate the configurations with real user profiles and real questions written by people, in place of synthetic data. The fifth direction is to tune the hyperparameters, such as the rank, the learning rate and the number of epochs, separately for each adapter type, and to test the explanations given in Section 4.4 for why LoRA did not increase the share of accepted recommendations for Llama 3.2 3B. This includes evaluating the adapters without quantization, selecting checkpoints on a validation set that contains difficult queries, and measuring how closely the responses of each configuration follow the responses of Qwen 3.6 35B. Finally, a statistical test of how often the annotators accept the recommendations of each configuration would add a measure of recommendation quality to the measure of agreement used here.

## 6. Conclusions

This study compared two parameter-efficient fine-tuning methods, LoRA and $IA^3$, with prompt engineering for personalized point-of-interest recommendation in the city of Pula. Llama 3.1 8B and Llama 3.2 3B were used, which gave six configurations. Three annotators rated the recommendations of each configuration for 1521 test queries, without knowing which model produced them. The agreement of the annotators, measured with Fleiss' kappa, was between 0.863 and 0.904 for all six configurations, which is almost perfect agreement. The evaluation protocol therefore gave consistent judgments for every adaptation strategy.

All four adapter configurations had a higher kappa than the same model without an adapter, but the differences were small. Only LoRA on Llama 3.1 8B remained statistically significant after correction for multiple comparisons. Neither adapter type was consistently better than the other, and the same configuration did not differ significantly between the two foundation models.

The annotators agreed with the recommendations of the $IA^3$ configurations most often (77.7% and 74.8% of the ratings), and $IA^3$ trains about 160 to 170 times fewer parameters than LoRA. These counts were not tested statistically. Fleiss' kappa measures how consistently the annotators judged the recommendations, not the quality of the recommendations, so the study does not show which configuration gives the best recommendations. The study used synthetic profiles and queries, one small city, two Llama models and one training run for each adapter. These limitations and the planned future work are described in Section 5.

**Author Contributions:** Conceptualization, R.K., L.S. and S.B.Š.; methodology, L.S.; software, L.S.; validation, I.L. and S.B.Š.; formal analysis, R.K.; investigation, R.K. and L.S.; resources, R.K. and V.M.; data curation, R.K. and L.S.; writing—original draft preparation, R.K. and L.S.; writing—review and editing, S.B.Š., I.L., and V.M.; visualization, R.K.; supervision, S.B.Š. and I.L.; project administration, S.B.Š. and I.L.; funding acquisition, V.M. and I.L. All authors have read and agreed to the published version of the manuscript.

**Funding:** This work was (partially) supported by the EC Digital Europe Programme EDIH Adria 2.0 (101256325); SPIN projects IP.1.1.03.0120, IP.1.1.03.0158 and IP.1.1.03.0039; and NextGenerationEU University grants: uniri-iz-25-6, uniri-iz-25-220, IIP_010144, IIP_010136

**Data Availability Statement:** The original data presented in this study are openly available in a public repository at https://github.com/FIPU-SPIN/ls-rk-lora-tourism-recommender

**Acknowledgments:** This research was performed using the Advanced computing service provided by University of Zagreb University Computing Centre - SRCE.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Abbreviations

The following abbreviations are used in this manuscript:

| | |
|---|---|
| AdamW | Adaptive Moment Estimation with Decoupled Weight Decay |
| AI | Artificial Intelligence |
| API | Application Programming Interface |
| BERT | Bidirectional Encoder Representations from Transformers |
| bfloat16 | Brain Floating Point 16-bit Format |
| CI | Confidence Interval |
| EC | European Commission |
| EDIH | European Digital Innovation Hub |
| FAISS | Facebook AI Similarity Search |
| GPT | Generative Pre-trained Transformer |
| GPU | Graphics Processing Unit |
| $IA^3$ | Infused Adapter by Inhibiting and Amplifying Inner Activations |
| JSON | JavaScript Object Notation |
| LLM | Large Language Model |
| LoRA | Low-Rank Adaptation |
| NF4 | 4-bit NormalFloat |
| PEFT | Parameter-Efficient Fine-Tuning |
| POI | Point of Interest |
| RAG | Retrieval-Augmented Generation |
| SRCE | University Computing Centre (Croatian: Sveučilišni računski centar) |
| T5 | Text-to-Text Transfer Transformer |
| TRL | Transformer Reinforcement Learning |
| $\kappa$ | Fleiss' kappa |

## References


1. Yoo, H.; Li, T.W.; Kang, S.; Liu, Z.; Xu, C.; Qi, Q.; Tong, H. Continual Low-Rank Adapters for LLM-based Generative Recommender Systems. *arXiv e-prints* **2025**, pp. arXiv–2510.
2. Ravaut, M.; Zhang, H.; Xu, L.; Sun, A.; Liu, Y. Parameter-efficient conversational recommender system as a language processing task. In Proceedings of the Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), 2024, pp. 152–165.
3. Zhu, J.; Lin, J.; Dai, X.; Chen, B.; Shan, R.; Zhu, J.; Tang, R.; Yu, Y.; Zhang, W. Lifelong Personalized Low-Rank Adaptation of Large Language Models for Recommendation. *arXiv e-prints* **2024**, pp. arXiv–2408.
4. Kaur, K.; Gupta, V.; Chadha, M.; Shah, C. Efficient and responsible adaptation of large language models for robust top-k recommendations. *ACM Transactions on Recommender Systems* **2026**, *4*, 1–31.
5. Yang, Z.; Gao, H.; Gao, D.; Yang, L.; Yang, L.; Cai, X.; Ning, W.; Zhang, G. Mlora: Multi-domain low-rank adaptive network for ctr prediction. In Proceedings of the Proceedings of the 18th ACM Conference on Recommender Systems, 2024, pp. 287–297.
6. Elmoghazy, S.S.; Shouman, M.A.; Elminir, H.K.; Selim, G.E.I. Comparative analysis of methodologies and approaches in recommender systems utilizing large language models. *Artificial Intelligence Review* **2025**, *58*, 207.
7. Bao, K.; Zhang, J.; Zhang, Y.; Wang, W.; Feng, F.; He, X. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In Proceedings of the Proceedings of the 17th ACM conference on recommender systems, 2023, pp. 1007–1014.
8. Liu, P.; Zhang, L.; Gulla, J.A. Pre-train, prompt, and recommendation: A comprehensive survey of language modeling paradigm adaptations in recommender systems. *Transactions of the Association for Computational Linguistics* **2023**, *11*, 1553–1571.
9. Nawara, D.; Kashef, R. A comprehensive survey on LLM-powered recommender systems: from discriminative, generative to multi-modal paradigms. *IEEE Access* **2025**.
10. Shehmir, S.; Kashef, R. LLM4Rec: a comprehensive survey on the integration of large language models in recommender systems—approaches, applications and challenges. *Future Internet* **2025**, *17*, 252.
11. Chen, J.; Liu, Z.; Huang, X.; Wu, C.; Liu, Q.; Jiang, G.; Pu, Y.; Lei, Y.; Chen, X.; Wang, X.; et al. When large language models meet personalization: Perspectives of challenges and opportunities. *World wide web* **2024**, *27*, 42.
12. Ling, C.; Zhao, X.; Lu, J.; Deng, C.; Zheng, C.; Wang, J.; Chowdhury, T.; Li, Y.; Cui, H.; Zhang, X.; et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *ACM Computing Surveys* **2025**, *58*, 1–39.
13. Xu, L.; Zhang, J.; Li, B.; Wang, J.; Chen, S.; Zhao, W.X.; Wen, J.R. Tapping the potential of large language models as recommender systems: A comprehensive framework and empirical analysis. *ACM Transactions on Knowledge Discovery from Data* **2025**, *19*, 1–51.

14. Kim, S.; Hwang, H.; Shin, K. Personalized Parameter-Efficient Fine-Tuning of Foundation Models for Multimodal Recommendation. In Proceedings of the Proceedings of the ACM Web Conference 2026, 2026, pp. 6079–6090.
15. Zeng, J.; Qi, Y.; Li, H.; Li, C.; Lyu, Z.; Cui, L.; Bai, L. RAIE: Region-Aware Incremental Preference Editing with LoRA for LLM-based Recommendation. In Proceedings of the Proceedings of the ACM Web Conference 2026, 2026, pp. 4038–4046.
16. Guo, H.; Liu, Z.; Tang, C.; Zhang, X. An interactive framework for personalized navigation based on metacosmic cultural tourism and large model fine-tuning. *IEEE Access* **2025**.
17. Choi, Y.; Kim, S.; Bassole, Y.C.F.; Sung, Y. Enhanced retrieval-augmented generation using low-rank adaptation. *Applied Sciences* **2025**, *15*, 4425.
18. Gao, J.; Peng, P.; Claramunt, C.; Lu, F. Quality Assessment of Tourist Generated Contents: A Large Language Model Approach. In Proceedings of the International Symposium on Web and Wireless Geographical Information Systems. Springer, 2025, pp. 31–41.
19. Nikolaychuk, O.A.; Poddubniy, I.A. Analysis of reviews of tourist objects based on large language models. In Proceedings of the 2024 IEEE 3rd International Conference on Problems of Informatics, Electronics and Radio Engineering (PIERE). IEEE, 2024, pp. 1070–1074.
20. Huang, Y.; Ma, L.; Wang, Y. Multi-Layer Prompt Engineering for Intent Recognition in Travel Planning Assistants. *Applied Sciences* **2025**, *15*, 11442.
21. Fleiss, J.L. Measuring nominal scale agreement among many raters. *Psychological Bulletin* **1971**, *76*, 378–382. https://doi.org/10.1037/h0031619.
22. Landis, J.R.; Koch, G.G. The measurement of observer agreement for categorical data. *Biometrics* **1977**, *33*, 159–174. https://doi.org/10.2307/2529310.


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.